

Um banco que não perde dinheiro

Transferências entre servidores distintos sem contas descasadas

Identificação

Universidade de São Paulo (USP)

Instituto de Ciências Matemáticas e de Computação (ICMC)

Campus São Carlos, São Paulo, Brasil

Área: Sistemas Distribuídos / Computação Distribuída

Integrantes

Número USP	Nome completo
18404636	Jefferson Daniel Flores Montenegro
18514632	Cristhian Jesus Maylle Briceño
17819748	Paulo Sebastian Rojo Pinedo

Descrição do Sistema a ser Desenvolvido

O sistema implementa um banco distribuído simples, com no máximo 3 servidores (mínimo 2), que juntos mantêm uma única cópia lógica das contas dos clientes. A regra central continua a mesma: nenhuma operação pode criar ou destruir dinheiro, mesmo se um dos servidores cair. Para manter o projeto simples, o sistema usa um esquema de primário e réplica(s), em vez de um protocolo de consenso complexo com eleição entre vários nós.

1. Arquitetura do Sistema

- **No máximo 3 servidores, no mínimo 2:** todos guardam a mesma base de contas; não há divisão dos clientes entre grupos diferentes.
- **Um primário, um ou dois réplicas:** o primário recebe todas as operações de escrita (depósito, saque, transferência); os demais servidores são réplicas.
- **Replicação antes de confirmar:** o primário só confirma uma operação ao cliente depois de replicá-la para a(s) réplica(s), garantindo que o dado sobreviva mesmo se o primário cair logo depois.
- **Failover simples:** as réplicas monitoram o primário por heartbeat (ping periódico); se ele parar de responder, a próxima da lista assume automaticamente como novo primário.
- **Transferências sempre locais:** como todas as contas estão no mesmo grupo replicado, uma transferência é sempre uma operação local no primário, sem necessidade de protocolo de duas fases entre grupos diferentes.
- **Cliente com lista de servidores:** o cliente guarda o endereço dos 2 ou 3 servidores; se tentar um que não é mais o primário, tenta o próximo até encontrar o atual.

2. Stack Tecnológico

- **Linguagem:** Python (proposta), pela sintaxe simples e rapidez para implementar sockets/threads e testar o failover sem muito código repetitivo.
- **Comunicação:** API HTTP simples com Flask ou FastAPI, trocando mensagens em JSON, mais fácil de testar do que sockets crus.
- **Persistência local:** arquivo JSON ou SQLite por servidor, sem banco de dados gerenciado.
- **Concorrência:** threads da biblioteca padrão (threading) para atender requisições e o heartbeat ao mesmo tempo.

- **Testes de falha:** scripts que derrubam o processo do primário (kill) ou atrasam respostas, para validar a troca automática.
- **Observabilidade:** log em arquivo texto por servidor (timestamp, operação, resultado).

3. Modelagem dos Dados

Entidade	Atributos principais	Observações
Conta	id, saldo	saldo nunca negativo
Operação	id, tipo (depósito/saque/transferência), conta_origem, conta_destino, valor, timestamp, status	compõe o extrato (RF-05)
Servidor	id, papel atual (primário/réplica), endereço	usado para failover e para o painel (F-11)
Registro de Replicação	id_operação, confirmado_por (lista de servidores que já aplicaram)	garante que a operação sobreviva à queda do primário

4. Regras de Negócio

- O saldo de uma conta nunca pode ficar negativo.
- A soma de todos os saldos do sistema é invariante: nenhuma operação cria ou destrói dinheiro.
- Uma transferência só é considerada concluída quando as duas pontas (débito e crédito) foram aplicadas; caso contrário, nenhuma é aplicada.
- Uma escrita só é confirmada ao cliente depois de replicada em pelo menos um outro servidor.
- Servidores podem cair ou ficar lentos, mas nunca agem de forma maliciosa (falhas por colapso, não bizantinas).
- Todas as contas ficam no mesmo grupo replicado; não há divisão entre servidores diferentes.
- Fora de escopo: autenticação de usuários, criptografia e interface gráfica web.

5. Histórias de Usuário

1. Como cliente, quero criar uma conta com saldo inicial, para começar a usar o sistema.
2. Como cliente, quero consultar o saldo da minha conta, para saber quanto dinheiro tenho disponível.
3. Como cliente, quero depositar ou sacar valores, para movimentar meu dinheiro.
4. Como cliente, quero transferir dinheiro para outra conta, mesmo que o servidor primário mude no meio do caminho.
5. Como cliente, quero consultar o extrato das minhas operações, para acompanhar meu histórico.
6. Como administrador, quero auditar o total de dinheiro em circulação, para garantir que nenhuma operação criou ou destruiu saldo.
7. Como cliente, quero que operações concorrentes com outros clientes sejam tratadas corretamente, para que meu saldo nunca fique inconsistente.
8. Como cliente, quero continuar usando o sistema mesmo se o servidor primário cair, para não depender de uma única máquina.
9. Como administrador, quero que um servidor recupere seu estado ao reiniciar, para não perder dados nem precisar de intervenção manual.
10. Como engenheiro de testes, quero derrubar o servidor primário propositalmente, para validar que a troca automática funciona.
11. Como administrador, quero visualizar quais dos 2 ou 3 servidores estão ativos e qual é o primário atual, para monitorar a saúde do sistema.
12. Como cliente, quero um cliente de linha de comando com acesso a todas as operações, para interagir sem depender de interface gráfica.

6. Tratamento de Concorrência

- Como existe um único primário, todas as escritas passam por ele e são processadas em ordem, o que já evita boa parte das condições de corrida sem mecanismos sofisticados.
- Para evitar que duas requisições concorrentes mexam na mesma conta ao mesmo tempo, o primário usa um lock simples por conta antes de aplicar a operação e replicá-la.
- Leituras podem ser atendidas pelo primário (sempre atualizado) ou, opcionalmente, pelas réplicas, aceitando que possam estar alguns milissegundos atrasadas.

7. Tolerância a Falhas: o Desafio do Projeto

O desafio central é simples de enunciar, mas importante de testar bem: o sistema não pode perder nem duplicar dinheiro quando o servidor primário cai.

- **Failover automático:** se o primário não responde ao heartbeat dentro de um tempo definido, a próxima réplica da lista assume, sem intervenção manual.
- **Replicação antes da confirmação:** nenhuma escrita é confirmada ao cliente antes de estar replicada em pelo menos um outro servidor.
- **Tolerância proporcional ao número de servidores:** com 2 servidores, o sistema tolera a queda de 1; com 3, tolera a queda de até 2 ao mesmo tempo (desde que um continue de pé).
- **Validação prática:** testes que derrubam o primário e atrasam mensagens comprovam que o failover funciona e que nenhum dinheiro é perdido.

8. Requisitos Funcionais (RF)

ID	Requisito	Prioridade
RF-01	Permitir criar contas com saldo inicial definido	Alta
RF-02	Permitir consultar o saldo de qualquer conta	Alta
RF-03	Permitir depósito e saque em uma conta	Alta
RF-04	Permitir transferência entre contas	Alta
RF-05	Garantir que uma transferência seja atômica: ou os dois lados são aplicados, ou nenhum	Alta
RF-06	Rejeitar transferências que deixariam o saldo negativo	Alta
RF-07	Garantir que uma leitura enxergue um estado consistente	Alta
RF-08	Fazer operações concorrentes sobre a mesma conta entrarem em conflito, sem corromper o saldo	Alta
RF-09	Eleger automaticamente um novo primário quando o atual falha	Alta
RF-10	Confirmar uma escrita ao cliente somente após replicação em pelo menos um outro servidor	Alta
RF-11	Continuar atendendo mesmo com um dos servidores fora do ar	Alta
RF-12	Recuperar o estado de um servidor reiniciado e reintegrá-lo ao sistema	Alta
RF-13	Resolver automaticamente operações interrompidas por falha do cliente ou do primário	Alta
RF-14	Oferecer uma auditoria do total de dinheiro em circulação	Alta
RF-15	Expor métricas de desempenho e estado dos servidores	Média
RF-16	Permitir injetar falhas durante a execução para fins de teste	Média
RF-17	Dar, via cliente de linha de comando, acesso a todas as operações	Média
RF-18	Permitir configurar, na inicialização, quais são os 2 ou 3 servidores participantes	Baixa

9. Requisitos Não Funcionais (RNF)

ID	Requisito	Critério de aceitação
RNF-01	Corretude sob falhas	O total de dinheiro nunca muda, em nenhum teste
RNF-02	Durabilidade	Operação confirmada sobrevive à queda do primário
RNF-03	Disponibilidade	Novo primário assume em poucos segundos após a falha
RNF-04	Desempenho	Ao menos 500 transações por segundo no ambiente local

ID	Requisito	Critério de aceitação
RNF-05	Latência	Transferência com p99 abaixo de 200 ms sem falhas
RNF-06	Testes reproduzíveis	Mesma semente de execução produz o mesmo resultado
RNF-07	Observabilidade	Log estruturado em todos os servidores
RNF-08	Modularidade	Cada componente com código, testes e documentação próprios
RNF-09	Portabilidade	Execução com um único comando em Linux e macOS
RNF-10	Documentação	Cada componente documenta seu funcionamento e seus modos de falha
RNF-11	Disponibilidade	Sistema continua respondendo enquanto pelo menos um servidor (de 2 ou 3) estiver ativo